

Full Length Article

SDCA: Towards semantic-guided dual camouflage for deceiving human eyes and object detectors

Haoqin Yuan^{a,b}, Xianyi Chen^{a,c,b,*}, Qi Cui^{id a,b}, Fazhan Liu^{a,b}, Zhangjie Fu^{id a,b}^a School of Cyber Science and Engineering, Nanjing University of Information Science and Technology, Nanjing, 210044, China^b Engineering Research Center of Digital Forensics, Ministry of Education, Nanjing University of Information Science and Technology, Nanjing, 210044, China^c Jiangsu Yuchi Blockchain Technology Research Institute Co., Ltd., Nanjing, 210018, China

ARTICLE INFO

Keywords:

Object detection

Camouflage

Adversarial attack

Perlin noise

ABSTRACT

Adversarial camouflage has gained widespread attention for its ability to prevent object detectors from accurately identifying physical-world targets. However, existing methods typically initialize textures with random values and exclude perturbation constraints during optimization. This approach lacks explicit guidance for generating perturbation patterns that conform to natural texture semantics (e.g., color distributions and contour structures), making the perturbations easily detected by biological vision systems. Regarding these problems, we propose Semantic-guided Dual Camouflage Attack (SDCA) from the perspective of joint optimization of natural semantics and adversarial perturbations. The core of SDCA consists of the Semantic-Driven Generator (SDG) and the Semantic-Constrained Optimization (SCO) strategy. SDG uses procedural noise to inversely model visual features of natural textures to achieve semantically-driven texture initialization. Meanwhile, SCO constrains the perturbation based on prior semantic information, preserving semantic consistency between the adversarial texture and the initial texture. Ultimately, SDCA can generate highly natural camouflage textures, achieving dual evasion of both biological vision systems and computer vision models. Experimental results on various detection tasks show that SDCA outperforms existing works in terms of naturalness while maintaining competitive robustness and transferability. The datasets and visualizations of SDCA are available at: <https://github.com/HaoqinYuan/Semantic-guided-Dual-Camouflage-Attack>.

1. Introduction

Camouflage is a multidimensional technical approach that achieves target concealment or adversary deception through visual disguise, auditory interference, and behavioral mimicry. This capability makes it critical for military, ecological, and industrial applications. Visual camouflage is an essential branch of camouflage that makes an object visually imperceptible by creating a special texture on its surface. Its application spans natural evolution and artificial design, manifested as survival strategies like protective coloration in biology, and extended to deception methods such as military camouflage in engineering.

The evolution of artificial vision camouflage is closely linked to advances in detection methods, leading to two distinct technical paths: anti-biological vision camouflage and anti-computer vision camouflage. These two approaches differ fundamentally in their core objectives, design principles, and key challenges.

Anti-biological vision camouflage (Pei et al., 2023; Xue et al., 2016; Yang et al., 2020) primarily focuses on deceiving biological vision systems. These approaches blur the boundary between foreground targets

and background environments by generating natural patterns with semantic features such as colors and contours of background elements (Cuthill, 2019). This type of camouflage aims to maximize texture naturalness. Its design is based on the perceptual characteristics of biological vision systems, focusing on spatial frequency, color adaptation, and contour integrity to achieve effective concealment in natural environments. The key challenge lies in accurately adapting to complex natural backgrounds while maintaining robust concealment across different scales. This places high demands on the natural appearance and adaptability of textures.

With the proliferation of deep neural networks (DNNs), the widespread adoption of object detection models (Jiang et al., 2022; Ren et al., 2016; Tan et al., 2020) has posed new challenges to traditional methods. These challenges have motivated researchers to approach the problem with adversarial attacks, promoting the development of adversarial camouflage (hereafter referred to as anti-computer vision camouflage Suryanto et al., 2023; Wang et al., 2022; Zhou et al., 2024b). The goal of anti-computer vision camouflage is primarily to maximize the attack performance of perturbations. This approach often employs

* Corresponding author.

E-mail address: xianyi_chen@nuist.edu.cn (X. Chen).<https://doi.org/10.1016/j.neunet.2026.108946>

Received 15 May 2025; Received in revised form 3 November 2025; Accepted 4 April 2026

Available online 11 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

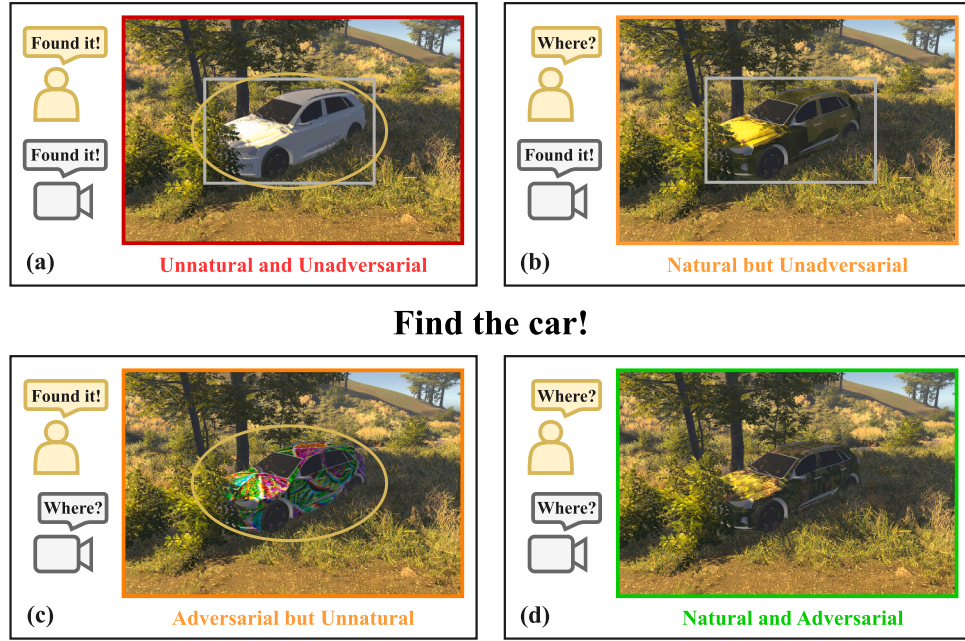


Fig. 1. (a) An uncamouflaged car; (b) The car employing anti-biological vision camouflage (e.g., DeCAM, Yang et al., 2023), struggling to evade object detectors; (c) The car employing anti-computer vision camouflage (e.g., RAUCA-final, Zhou et al., 2024b), exhibiting suspicious visual patterns that are easily detectable by human eyes; (d) The car camouflaged by SDCA, successfully deceiving human eyes and object detectors.

gradient-based optimization strategies, which leverage gradient information to actively obtain perturbations capable of crossing the object detector’s decision boundaries. However, due to the “black-box” nature and complexity of DNNs, perturbations optimized for attack strength often lack interpretability, posing a significant challenge in achieving generalization in unknown environments.

To achieve their respective design goals and address key challenges, both anti-biological and anti-computer vision camouflage have developed distinct technical approaches. Yet, each has revealed certain limitations. To enhance texture naturalness, anti-biological vision camouflage is usually colorized according to the scene’s dominant colors, and its texture is usually constructed based on pre-generated templates (Pei et al., 2023; Wang et al., 2023) or natural patterns (Yang et al., 2020, 2023). This mechanical approach struggles to create effective adversarial perturbations, which limits the attack performance of the anti-biological vision camouflage and makes it challenging to evade detection by computer vision models, as shown in Fig. 1(b). Correspondingly, to enhance attack performance, most existing anti-computer vision camouflage methods initialize textures with random values (Wang et al., 2022; Zhou et al., 2024a,b). This approach allows optimization to start from an unstructured, semantically-weak initial point, resulting in texture patterns exhibiting non-deterministic distribution characteristics at the outset. During optimization, to explore a broader attack space, most methods exclude perturbation constraints, exacerbating the uncertainty of the texture structure. Due to the nonlinearity and complexity of DNNs, such uncertainty causes the final generated textures to deviate from the natural semantic distribution. While this may enhance attack performance to some extent, it simultaneously increases the likelihood of detection by biological vision systems, as shown in Fig. 1(c).

To address these issues, some studies have begun exploring methods that balance naturalness and attack performance. DAAC (Wang et al., 2023) embeds adversarial patches into pre-generated anti-biological vision camouflage textures, while DAC (Sun et al., 2023) superimposes independently optimized pixel-level perturbations onto initial textures that match the natural background. These methods achieve dual evasion of human eyes and object detectors by reducing perturbation visibil-

ity. However, their completely decoupled multi-stage processing strategy prevents the perturbation generation stage from utilizing feature information from the semantic construction stage. Consequently, this lack of synergy between the two stages ultimately limits the camouflage effectiveness.

To break through the performance bottleneck of existing methods, we propose Semantic-guided Dual Camouflage Attack (SDCA), a novel anti-computer vision camouflage generation framework. Its core innovation lies in establishing bidirectional interaction between natural semantics and adversarial perturbations, thereby enabling semantic-guided end-to-end generation of camouflage textures. This approach significantly enhances naturalness while maintaining attack performance, as shown in Fig. 1(d).

Our contributions can be summarized as follows.

- We propose the Semantic-Driven Generator (SDG), a specialized initialization component that constructs natural semantic features to generate realistic initial textures, thus establishing an informative prior for subsequent optimization tasks.
- We propose the Semantic-Constrained Optimization (SCO), an advanced optimization strategy that generates a unique semantic perturbation constrained by prior semantic knowledge, ensuring semantic consistency between the adversarial texture and the initial texture.
- Our method enables effective control over the perturbation pattern without stacking complex loss terms, improving texture generation efficiency.
- We designed a dual-dimensional evaluation protocol that establishes quantifiable metrics for attack performance and naturalness. Extensive experiments demonstrate that our method achieves superior naturalness while maintaining competitive robustness and transferability.

2. Related work

In this section, we review anti-biological vision camouflage and anti-computer vision camouflage successively.

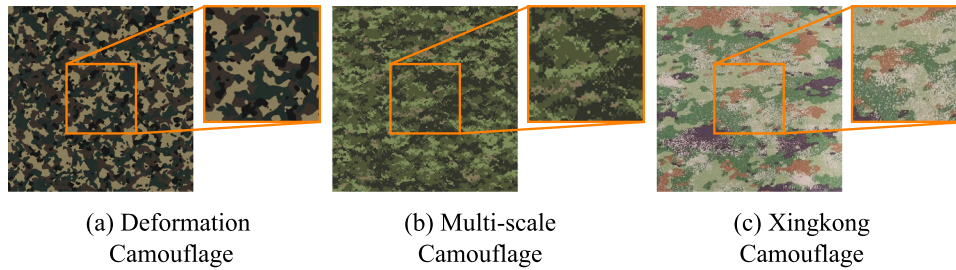


Fig. 2. Typical artificial anti-biological vision camouflage.

2.1. Anti-biological vision camouflage

Anti-biological vision camouflage disrupts biological vision systems by mimicking natural semantic features such as color distributions and contour structures. The primitive form of such camouflage is the “protective coloration” in nature, which helps organisms to evade predators (Caro & Koneru, 2021). With the deepening of research on biological camouflage mechanisms and the increasing demand for practical applications, such as military defense, researchers have begun to explore artificial design methods for anti-biological vision camouflage. Such camouflage can be classified into three categories based on textural appearance characteristics: Deformation camouflage, Multi-scale camouflage,¹ and Xingkong camouflage².

Deformation camouflage is a speckle texture, typically composed of irregular spot patterns (as shown in Fig. 2(a)). Common generation methods fall into two categories: (1) Using procedural noise to simulate natural patterns. For instance, Perlin Noise (Perlin, 1985) generates a continuous noise map by gradient interpolation, effectively simulating smooth edges. Worley Noise (Worley, 1996), on the other hand, generates discrete cellular structures based on Voronoi diagrams (Aurenhammer, 1991) and is suitable for constructing hard-edged patches. (2) Employing Generative Adversarial Networks (GANs) to learn texture features from the environment. While Yang et al. (2020) proposed a method to generate single speckle patterns, they later introduced a background matching strategy to synthesize complete camouflage textures (Yang et al., 2023), overcoming the limitation of isolated patterns being incompatible with global backgrounds. While deformation camouflage provides effective concealment at close ranges, its effectiveness declines with increasing observation distance.

Multi-scale camouflage is a pixelated texture, usually consisting of an arrangement of pre-generated polygonal templates (as shown in Fig. 2(b)). For instance, Xue et al. (2016) adopted an iterative superposition strategy to dynamically adjust the layout of irregular templates. Jia et al. (2020) enhanced texture hierarchy by superimposing templates of varying dimensions. Wang et al. (2023) constructed the camouflage pattern by randomly arranging quadrilateral templates. Pei et al. (2023), on the other hand, used the lowest horizontal line algorithm to arrange hexagonal templates, producing more complex and adaptive pattern structures. High-frequency template elements (e.g., small-scale edges or dots) may visually blend into macroscopic color blocks when viewed from a distance, enabling multi-scale concealment by combining high-frequency and low-frequency features. However, rigid geometric structures of the templates inherently restrict the diversity of generated texture patterns.

Xingkong camouflage is a composite texture, featuring a speckle texture as the base layer and a starry texture composed of discrete pixels as the top layer (as shown in Fig. 2(c)). The fusion of multiple semantic features enables it to simulate a wide range of natural textures, providing enhanced concealment capabilities across various environments

(Army Recognition, 2019). Therefore, we construct natural semantics based on Xingkong camouflage patterns to maximize the evasion performance of camouflage textures against biological vision systems. However, due to the incomplete theoretical research on composite texture generation methods, existing studies on Xingkong camouflage remain limited. To achieve inverse modeling of this pattern, we employ procedural noise to construct composite textures, laying the foundation for the subsequent generation of adversarial textures.

2.2. Anti-computer vision camouflage

Anti-computer vision camouflage facilitates adversarial attacks by optimizing the texture on the object’s surface. This type of camouflage can be applied to various targets, including human bodies and vehicles. With the rapid advancement of technologies such as vehicle detection (Abiyev & Arslan, 2023) and autonomous driving (Chiu et al., 2025), most studies focus on vehicles as the research subject.

Given the non-planar nature of object surfaces, existing methods use three-dimensional (3D) renderers to map textures on complex geometric surfaces accurately. Traditional renderers (e.g., Unreal Engine³, Unity3D⁴) involve a series of discrete operations during rendering, including rasterization and visibility determination. This non-differentiable process prevents neural networks from performing end-to-end training directly through backpropagation. To address this issue, CAMOU (Zhang et al., 2019) proposed a black-box approach that employs a neural approximation function to simulate the rendering process, thereby generating adversarial textures. However, due to the inherent approximation errors, its performance is generally inferior to white-box-based methods.

To overcome the technical limitations in integrating traditional renderers with neural networks, Kato et al. (2018) proposed Neural Renderer. This approach employs approximate gradients to smooth discrete operations in the rendering pipeline, thereby enabling backpropagation of rendering parameters. Leveraging the differentiability of Neural Renderer, DAS (Wang et al., 2021) developed a white-box attack method based on dual attention suppression, which addresses the limitations of black-box attacks. However, this method only generates adversarial patches on specific parts of the vehicle, resulting in limited cross-view attack performance. Consequently, FCA (Wang et al., 2022) introduced a full-coverage attack approach that generates adversarial textures across the entire vehicle surface.

With the continuous development and wide application of neural rendering, many researchers have focused on improving the attack performance (e.g., robustness, transferability, and stability). To enhance robustness under varying lighting conditions, DTA (Suryanto et al., 2022) utilized a differential transform network to obtain realistic rendered images. To enhance cross-scene applicability, ACTIVE (Suryanto et al., 2023) employs a tri-planar mapping technique to improve the transferability of camouflage textures among different vehicle models. For com-

¹ https://en.wikipedia.org/wiki/Multi-scale_camouflage

² [https://en.wikipedia.org/wiki/Xingkong_\(camouflage\)](https://en.wikipedia.org/wiki/Xingkong_(camouflage))

³ <https://www.unrealengine.com>

⁴ <https://unity.com>

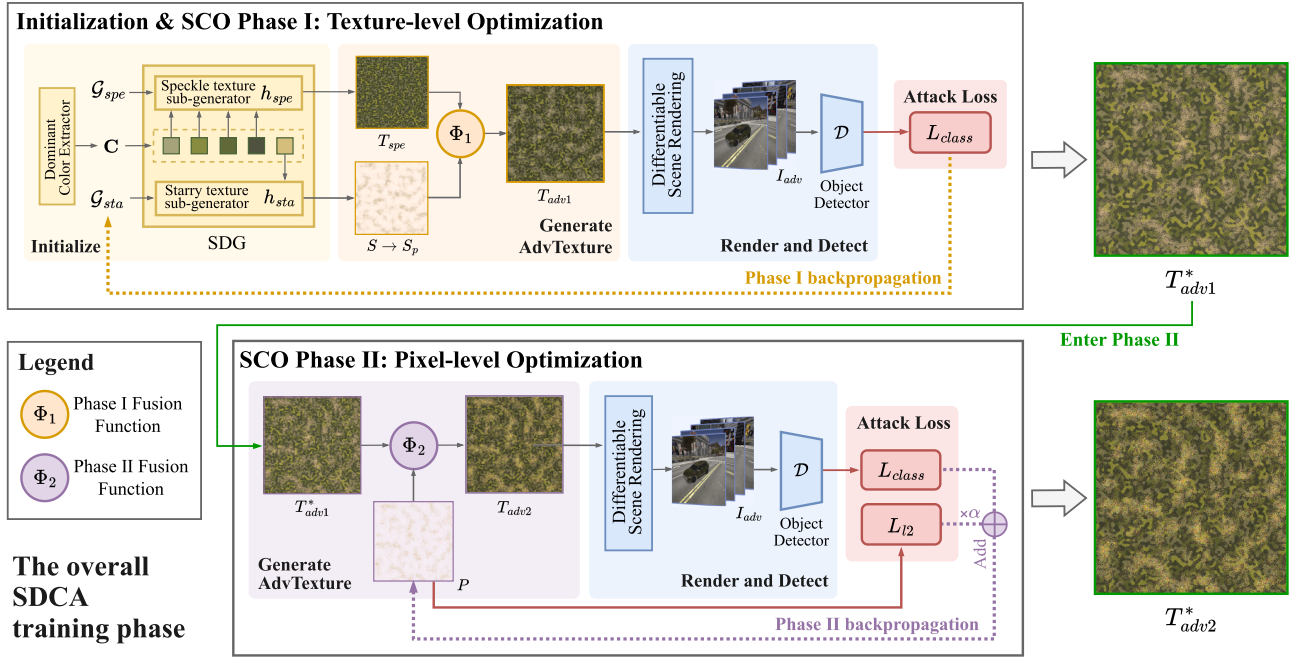


Fig. 3. The overview of the SDCA framework. We first utilize the Semantic-Driven Generator (SDG) to generate the initial texture that conforms to natural semantics. Then, we follow the Semantic-Constrained Optimization (SCO) strategy to optimize the texture under the constraint of prior semantic information.

plex weather challenges, RAUCA (Zhou et al., 2024a,b) uses an end-to-end neural renderer to simulate the environmental characteristics under various weather conditions. For multi-view stability requirements, MFA (Li et al., 2024) proposes a neural fusion renderer to achieve more consistent attack effects.

However, existing methods mainly focus on optimizing attack performance while neglecting texture naturalness. This limitation manifests in two key aspects: First, most approaches employ random noise (Wang et al., 2022; Zhou et al., 2024a,b) or colored pixel blocks (Suryanto et al., 2022; Zhang et al., 2019) to initialize. This approach generates unnatural results, often exhibiting overly vivid colors or fragmented structures. Secondly, although some methods attempt to incorporate regularization terms (Suryanto et al., 2023; Zhou et al., 2024a,b) to constrain the perturbation, these constraints typically operate on either local features or single-dimensional attributes (e.g., color or structure), failing to control the overall pattern of the texture. In conclusion, all these methods cannot realize the active guidance of the perturbation pattern and only passively limit the intensity of the perturbation. To break through this limitation, we utilize natural semantics in anti-biological vision camouflage to guide the generation of the anti-computer vision camouflage: (1) constructing initial textures containing natural semantics by procedural noise, and (2) optimizing the perturbations under the constraints of the prior semantic information.

3. Method

In this section, we first provide an overview of the proposed approach, then define the problem model, and finally elaborate on the implementation details.

3.1. Overview of SDCA

To address the issue of existing anti-computer vision camouflage methods generating textures that deviate from natural semantic distributions, we propose Semantic-guided Dual Camouflage Attack (SDCA), which innovatively improves both the initialization and optimization stages, as illustrated in Fig. 3.

In the initialization stage, we first explicitly model the semantic features of natural textures (color and contour). We employ a dominant color extractor (Section 3.3.1) to extract representative color features from natural scene images. To reduce interference from irrelevant low-frequency regions like sky and shadows, we concentrate on high-frequency areas in the image and cluster pixel values in the Hue-Saturation-Value (HSV) color space. This approach ensures our results maintain alignment with human visual perception.

Next, we employ the Semantic-Driven Generator (SDG, Section 3.3.2) to construct contour features and fuse them with the pre-extracted color features. To enhance the biological vision evasion capability of the initial texture, SDG adopts an inverse modeling approach using Xingkong camouflage as the design target. Xingkong camouflage enhances the full-area concealment performance by combining: (1) an underlying speckle texture T_{spe} that composes overlapping irregular speckle patterns, and (2) an overlying starry texture S that is essentially a discrete point set exhibiting multi-scale statistical characteristics similar to stellar distributions in astronomical observations. To model these textures, SDG utilizes two specialized sub-generators: the speckle texture sub-generator h_{spe} and the starry texture sub-generator h_{sta} . Both sub-generators are implemented using Perlin noise (Perlin, 1985), chosen for its continuous properties that enable: (1) Simulation of smooth transitions in speckle textures. (2) Reproduction of natural distributions in starry patterns.

Specifically, h_{spe} generates speckle patterns individually based on the noise maps at each scale, then forms the speckle texture through multi-scale superposition. This approach effectively disrupts the object's boundary edges and interferes with human visual perception across different frequency ranges. Meanwhile, h_{sta} simulates the complex distribution of the starry texture by superimposing noise maps with multiple scales. It incorporates an adaptive filtering strategy to preserve original semantic information when converting the noise map into a discrete point set.

To ensure semantic consistency between adversarial textures and initial textures during optimization, we adopt the Semantic-Constrained Optimization (SCO, Section 3.3.4) strategy. This approach generates the adversarial texture under the constraints of prior semantic information, while maintaining its attack effectiveness across varying

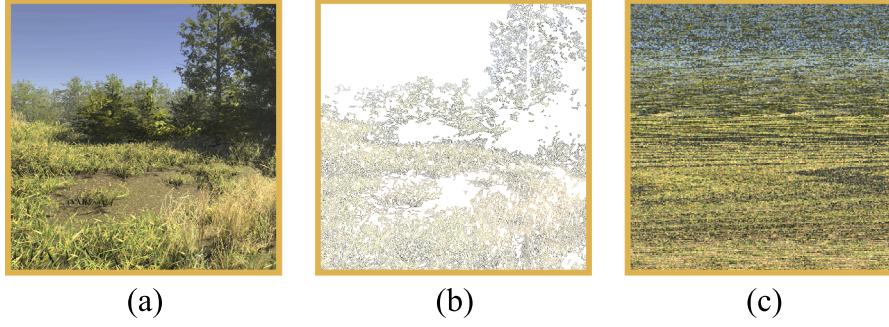


Fig. 4. Extraction process.

distances. Most previous works (Wang et al., 2022; Zhou et al., 2024a,b) modified the entire texture of the object, resulting in continuous and conspicuous perturbations. To reduce the perception of perturbations by biological vision systems, we optimize the discrete point set rather than the entire adversarial texture. The discrete point set contains substantial high-frequency information, which delays visual response time (Shulman et al., 1986), and its semi-transparent design further reduces the visibility of suspicious pixels.

The SCO strategy formulates the generation of the adversarial texture as an optimization problem, which is decomposed into two stages: texture-level optimization and pixel-level optimization. In the first phase, we form the semantic perturbation S_p by adjusting the distribution of the discrete point set in S . This perturbation is always constrained within the gradient field of Perlin noise, achieving close-range attacks while maintaining semantic consistency. When the detection distance increases, the distribution details of the discrete set are lost due to downsampling, leading to degraded texture-level attack performance. To address this limitation, we introduce pixel-level optimization that compensates for the attack effectiveness by adaptively adjusting the pixel values of S_p .

3.2. Problem definition

For an object to be camouflaged, we aim to generate an adversarial texture on its surface that prevents the detector from recognizing or localizing it. This process mainly consists of two core aspects: differentiable scene rendering and adversarial attack.

Differentiable scene rendering is achieved through the following steps: First, a rendered image of the camouflaged object $O_{adv} = \mathcal{R}(M, T_{adv}, \lambda)$ is generated, where $\mathcal{R}$ denotes the neural renderer, M represents the 3D mesh of the object, T_{adv} is the object's texture, and λ is the camera parameters (e.g., viewpoint, distance). Next, a sampled image of the camouflaged object $I_{adv} = \mathcal{F}(O_{adv})$ is obtained by transferring the object into diverse simulated scenes via a transformation function $\mathcal{F}$.

In the camouflage attack stage, we first employ the Semantic-Driven Generator (SDG) to generate the speckle texture T_{spe} and the starry texture S , as follows:

$$T_{spe} = h_{spe}(\mathcal{G}_{spe}) \quad (1)$$

$$S = h_{sta}(\mathcal{G}_{sta}) \quad (2)$$

where $\mathcal{G}_{spe}$ and $\mathcal{G}_{sta}$ represent the input parameter sets for Perlin noise. Following the Semantic-Constrained Optimization (SCO) strategy, we then optimize the initial texture. During the texture-level optimization phase, leveraging the deterministic nature of Perlin noise, we achieve controlled fine-tuning of the global distribution of the discrete point set in S by adjusting the gradient set $\mathcal{G}_{sta}$, thereby forming the semantic perturbation S_p . The adversarial texture T_{adv1} in phase one is generated by the fusion function Φ_1 and optimized by minimizing the loss function L_{phase1} , yielding the intermediate result of the whole optimization

process T_{adv1}^* . The objective function is as follows:

$$T_{adv1} = \Phi_1(T_{spe}, h_{sta}(\mathcal{G}_{sta})) \quad (3)$$

$$T_{adv1}^* = \arg \min_{\mathcal{G}_{sta}} \mathcal{L}_{phase1}(D(\mathcal{F}(\mathcal{R}(M, T_{adv1}, \lambda))), y) \quad (4)$$

where y is the ground truth label. During the pixel-level optimization phase, we define a pixel perturbation P to adjust the pixel values of the semantic perturbation S_p . Then, the adversarial texture T_{adv2} in phase two is generated by the fusion function Φ_2 . The final optimized texture T_{adv2}^* is obtained by minimizing the loss function L_{phase2} , which is formulated as:

$$T_{adv2} = \Phi_2(T_{adv1}^*, P) \quad (5)$$

$$T_{adv2}^* = \arg \min_P \mathcal{L}_{phase2}(D(\mathcal{F}(\mathcal{R}(M, T_{adv2}, \lambda))), y) \quad (6)$$

3.3. Detailed implementation of SDCA

3.3.1. Dominant color extractor

For a natural background image (Fig. 4(a)), the dominant color extractor first employs Canny edge detection (Rong et al., 2014) to identify high-frequency regions (Fig. 4(b)), then merges the pixels within these regions into a composite image (Fig. 4(c)), thereby achieving the removal of most low-frequency areas. Next, the pixel values in the composite image are clustered using the K-means algorithm, with a customized distance metric d in the HSV color space:

$$d(n_i, n_j) = \sqrt{\left(\frac{\min(\Delta h, 360 - \Delta h)}{360}\right)^2 + \Delta s^2 + \Delta v^2} \quad (7)$$

where for each pixel n_k ($k \in \{i, j\}$), $h_k \in [0^\circ, 360^\circ]$, $s_k \in [0, 1]$, and $v_k \in [0, 1]$ represents its hue, saturation, and value components, respectively, and $\Delta h = |h_i - h_j|$, $\Delta s = |s_i - s_j|$, $\Delta v = |v_i - v_j|$. Since Hue is a circular angle on the color wheel, we use $\min(\Delta h, 360 - \Delta h)$ to compute the shortest angular difference. To further minimize the influence of irrelevant pixels on clustering results, we set the number of clusters to 6 and extract the HSV components of the centroids from the top five largest clusters (by pixel count). Following Smith (1978), we convert the HSV components to Red-Green-Blue (RGB) components for subsequent initialization, obtaining the dominant color set $C = \{C_i\}_{i=1}^5$, where $C_i = (r_i, g_i, b_i) \in [0, 1]^3$.

3.3.2. Semantic-driven generator (SDG)

This module uses the following process to generate textures (as shown in Fig. 5): First, noise maps are generated by the Perlin noise algorithm, followed by post-processing operations (e.g., binarization, multi-scale superposition, and adaptive filtering) to convert noise maps into contour feature maps. Finally, the contour and color features are fused using a layered coloring strategy adapted from Pei et al. (2023) and Wang et al. (2023), applying four dominant colors $\{C_i\}_{i=1}^4$ to the speckle texture while reserving the high-contrast C_5 for the starry texture.

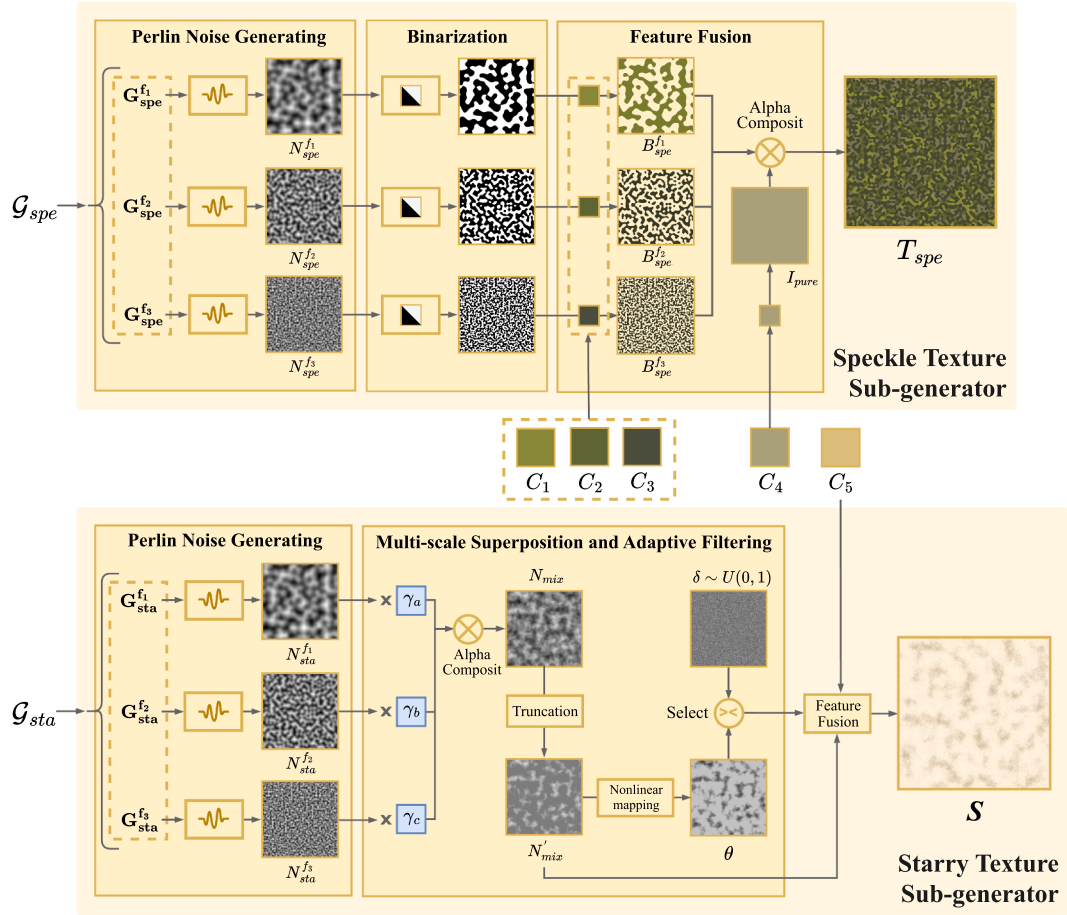


Fig. 5. SDG architecture. This module consists of the speckle texture sub-generator and the starry texture sub-generator to achieve hierarchical modeling of the initial texture.

Perlin noise generation. In this paper, we follow [Gustavson \(2005\)](#) to generate noise maps. First, a base grid of size $f \times f$ is initialized based on the scale factor f . Each grid point (i, j) is randomly assigned a gradient vector $\mathbf{g}_{i,j} = (g_x, g_y)$, forming a gradient matrix:

$$\mathbf{G}^f = [\mathbf{g}_{i,j}]_{(f+1) \times (f+1)} \quad (8)$$

Next, convolution and interpolation ([Lagae et al., 2010](#)) are applied to these gradient values to produce a smooth noise map N^f . This single-channel image contains normalized values in the range $[0, 1]$, functionally equivalent to a floating-point grayscale image. Note that the size of a noise map is arbitrary and independent of its scale. The scale factor f controls the frequency (level of detail) rather than the dimensions of the noise map.

Speckle texture sub-generator. This sub-generator first generates noise maps $\{N_{spe}^{f_k}\}_{k=1}^3$ with dimensions $l_1 \times l_1$ according to the gradient matrix set $\mathcal{G}_{spe} = \{\mathbf{G}_{spe}^{f_k}\}_{k=1}^3$, where the factors f_1, f_2, f_3 correspond to large, medium, and small scales respectively. These noise maps are then binarized and colored to obtain speckle patterns $\{B_{spe}^{f_k}\}_{k=1}^3$, where each $B_{spe}^{f_k} \in [0, 1]^{l_1 \times l_1 \times 4}$ denotes a normalized Red-Green-Blue-Alpha (RGBA) image:

$$B_{spe}^{f_k}(i, j) = \begin{cases} (r_{t(k)}, g_{t(k)}, b_{t(k)}, 1) & \text{if } N_{spe}^{f_k}(i, j) \geq \epsilon_{pixel} \\ (0, 0, 0, 0) & \text{if } N_{spe}^{f_k}(i, j) < \epsilon_{pixel} \end{cases} \quad (9)$$

where $t(k) : \{1, 2, 3\} \rightarrow \{1, 2, 3\}$ maps the scale index k to the dominant color index $t \in \{1, 2, 3\} \subset \{1, \dots, 5\}$, $\epsilon_{pixel} \in [0, 1]$ is the gray-level threshold. Finally, the speckle texture $T_{spe} \in [0, 1]^{l_1 \times l_1 \times 4}$ is obtained by alpha-

compositing ([Porter & Duff, 1984](#)) the multi-scale speckle patterns with a solid color image $I_{pure} \in [0, 1]^{l_1 \times l_1 \times 4}$:

$$I_{pure}(i, j) = (r_4, g_4, b_4, 1) \quad (10)$$

$$T_{spe} = I_{pure} \odot B_{spe}^{f_1} \odot B_{spe}^{f_2} \odot B_{spe}^{f_3} \quad (11)$$

where $\odot$ is the over operator.

Starry texture sub-generator. Following [Ma et al. \(2025\)](#), this sub-generator first generates noise maps $\{N_{sta}^{f_k}\}_{k=1}^3$ with dimensions $l_2 \times l_2$ according to the gradient matrix set $\mathcal{G}_{sta} = \{\mathbf{G}_{sta}^{f_k}\}_{k=1}^3$. These noise maps are then superposed to obtain a fused noise map N_{mix} :

$$N_{mix} = \gamma_1 N_{sta}^{f_1} \odot \gamma_2 N_{sta}^{f_2} \odot \gamma_3 N_{sta}^{f_3} \quad (12)$$

where $\gamma_1, \gamma_2, \gamma_3$ are the fusion coefficients used to adjust the contribution ratios of noise at different scales. Subsequently, we designed an adaptive filtering strategy based on grayscale values to remove certain pixel points. Specifically, for each pixel (i, j) in the noise map, a random variable $\delta(i, j) \sim U(0, 1)$ is generated. The pixel is removed when the following condition is met:

$$\delta(i, j) < \theta(i, j) \quad (13)$$

where $\theta(i, j)$ represents the retention threshold. Based on the spatial correlation characteristics of stellar distributions, we establish a nonlinear mapping relationship between grayscale values and point density: high grayscale values correspond to point-dense regions. In contrast, low grayscale values correspond to point-sparse regions. Therefore, $\theta(i, j)$ is

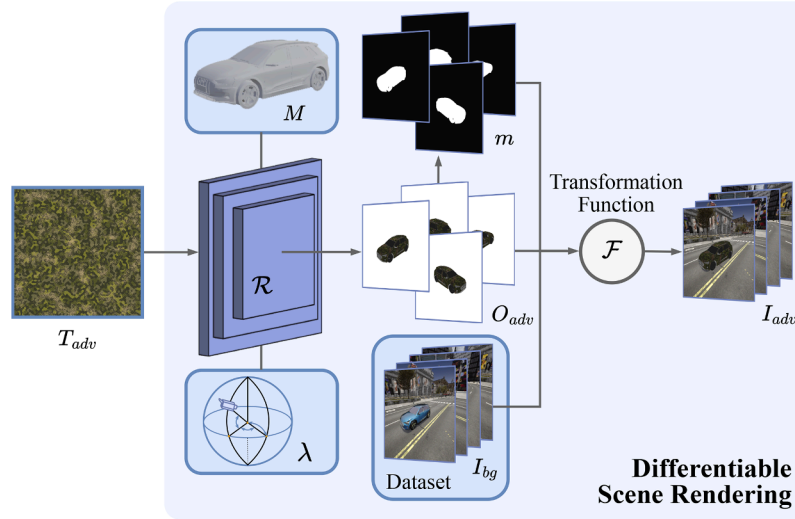


Fig. 6. Rendering process.

determined by the fused noise map N_{mix} :

$$N'_{mix}(i, j) = \max(N_{mix}(i, j), 0.5) \quad (14)$$

$$\theta(i, j) = e^{(1-N'_{mix}(i, j))^\sigma} - 1 \quad (15)$$

where N'_{mix} is the truncated noise map, ensuring uniform point distribution in point-sparse regions. The curvature parameter $\sigma \in [0.5, 1)$ adjusts the bending degree of the probability mapping curve, indirectly controlling the retention probability in point-dense regions (high values in N'_{mix}) and the removal intensity in point-sparse regions (low values in N'_{mix}). Finally, we remove all pixels that meet the removal condition and apply coloring to the retained pixels, obtaining the starry texture $S \in [0, 1]^{l_2 \times l_2 \times 4}$:

$$S(i, j) = \begin{cases} (r_5, g_5, b_5, N'_{mix}(i, j)) & \text{if } \delta(i, j) \geq \theta(i, j) \\ (0, 0, 0, 0) & \text{if } \delta(i, j) < \theta(i, j) \end{cases} \quad (16)$$

During colorization, we dynamically adjusted the transparency of the pixel points based on N'_{mix} to achieve a visually continuous representation of stellar density: point-dense regions exhibit higher opacity, while point-sparse regions gradually transition to transparency.

3.3.3. Differentiable scene rendering

Following Wang et al. (2022), we use a fusion function F to obtain the sampled image I_{adv} (as shown in Fig. 6):

$$I_{adv} = F(O_{adv}) = O_{adv} \cdot m + I_{bg} \cdot (1 - m) \quad (17)$$

where m is the binary mask generated from the rendered image O_{adv} , I_{bg} is the background image.

3.3.4. Semantic-constrained optimization (SCO)

Phase one: Texture-level optimization. In this phase, we obtain the semantic perturbation S_p by adjusting the gradient set G_{sta} . The adversarial texture T_{adv1} can be computed as follows:

$$T_{adv1} = \Phi_1(T_{spe}, h_{sta}(G_{sta})) = T_{spe} \odot \text{Resize}(h_{sta}(G_{sta})) \quad (18)$$

where $\text{Resize}(\cdot)$ scales S_p to ensure its dimensions match those of the speckle texture T_{spe} . To achieve the attack, we define the classification loss $\mathcal{L}_{class}$ to minimize the prediction probability of the target class. For a single image m , we first obtain its confidence score $\text{Conf}(m)_c$ for the target class c based on the original output $\text{logits}(m)$ of the model, as follows:

$$\text{NMS}(\text{logits}(m), \epsilon_{conf}, \epsilon_{IOU}) = \{x^k, y^k, w^k, h^k, \text{conf}^k, \text{cls}^k\}_{k=1}^K \quad (19)$$

$$\text{Conf}(m)_c = \sum_{k \in \{1, \dots, K\} | \text{cls}^k = c} \text{conf}^k \quad (20)$$

where $\text{NMS}(\cdot)$ is the Non-Maximum Suppression (NMS) function, ϵ_{conf} and ϵ_{IOU} are the confidence and Intersection Over Union (IOU) threshold, K is the total number of bounding boxes, and $(x^k, y^k), (w^k, h^k), \text{conf}^k, \text{cls}^k$ represent the location, size, confidence score and class label of the k th bounding box, respectively. Then, we sum up the confidence scores across all images in a batch to obtain $\mathcal{L}_{class}$, as follows:

$$\mathcal{L}_{class} = \sum_{b=1}^B \text{Conf}(m_b)_c \quad (21)$$

where B is the batch size, m_b is the b th sample.

Phase two: Pixel-level optimization. In this phase, we modify the pixel values of the semantic perturbation S_p using a pixel perturbation $P \in [-1, 1]^{l_2 \times l_2}$, and obtain the adversarial texture T_{adv2} as follows:

$$T_{adv2} = \Phi_2(T_{adv1}^*, P) = \text{Clip}(T_{adv1}^* + \text{Resize}(P)) \quad (22)$$

where $\text{Clip}(\cdot)$ is used to limit the pixel values to the normal range $(0, 1)$. To mitigate overfitting, we introduce the regularization loss $\mathcal{L}_{l_2}$ alongside the classification loss, as follows:

$$\mathcal{L}_{l_2} = \|P\|_2 = \sqrt{\sum_{i,j} P(i, j)^2} \quad (23)$$

In summary, the final adversarial loss $\mathcal{L}_{mix}$ in phase two is constructed as follows:

$$\mathcal{L}_{mix} = \mathcal{L}_{class} + \alpha \cdot \mathcal{L}_{l_2} \quad (24)$$

where α is the regularization coefficient.

The full pipeline of the SDCA framework is illustrated in Algorithm 1, which generates dual camouflage through a two-phase approach. The first phase performs texture-level optimization by minimizing $\mathcal{L}_{class}$, while the second phase optimizes at the pixel level by minimizing $\mathcal{L}_{mix}$.

4. Experiments

4.1. Experimental settings

4.1.1. Experimental design

Our experiments are structured into attack performance evaluation, naturalness evaluation, and ablation studies. Given that the primary objective of the anti-computer vision camouflage is to deceive object detectors, the first part of our experiments focuses on evaluating the

Algorithm 1 Two-phase dual camouflage generation of SDCA.**Input:**

- **Parameters:** Car model mesh M , parameter sets $\mathcal{G}_{spe}$ and $\mathcal{G}_{sta}$, pixel perturbation P , regularization coefficient α , number of iterations N .
- **Data:** Dataset $\mathcal{T} = \{(I_{bg}^j, \lambda^j)\}_{j=1}^{|\mathcal{T}|}$ of background images I_{bg} and camera parameters λ .
- **Functions:** Neural renderer $\mathcal{R}(\cdot, \cdot, \cdot)$, speckle texture sub-generator $h_{spe}(\cdot)$ and starry texture sub-generator $h_{sta}(\cdot)$ from Section 3.3.2, confidence score $Conf(\cdot)_c$ from Eq. (20).

Output: Dual camouflage T_{adv}^* .

Phase One $\rightarrow$ goal: obtaining the intermediate result T_{adv1}^* by adjusting $\mathcal{G}_{sta}$. <pre> 1: Random initialize $\mathcal{G}_{spe}, \mathcal{G}_{sta}$ 2: $T_{spe} \leftarrow h_{spe}(\mathcal{G}_{spe})$ 3: for $i \leftarrow 1$ to N do 4: $S_p \leftarrow h_{sta}(\mathcal{G}_{sta})$ 5: $T_{adv1} \leftarrow T_{spe} \odot \text{Resize}(S_p)$ 6: for $j \leftarrow 1$ to $\mathcal{T}$ do 7: $O_{adv}, m \leftarrow \mathcal{R}(M, T_{adv1}, \lambda^j)$ 8: $I_{adv} \leftarrow O_{adv}, m + I_{bg}^j (1 - m)$ 9: $\mathcal{L}_{class}^j \leftarrow Conf(I_{adv})_c$ 10: end for 11: $\mathcal{L}_{class} \leftarrow \sum_{j=1}^{ \mathcal{T} } \mathcal{L}_{class}^j$ 12: Update $\mathcal{G}_{sta}$ with $\nabla_{\mathcal{G}_{sta}} \cdot \mathcal{L}_{class}$ 13: end for 14: $S_p \leftarrow h_{sta}(\mathcal{G}_{sta})$ 15: $T_{adv1}^* \leftarrow T_{spe} \odot \text{Resize}(S_p)$ </pre>	Phase Two $\rightarrow$ goal: obtaining the final optimized texture T_{adv2}^* by adjusting P. <pre> 16: Zero initialize P 17: for $i \leftarrow 1$ to N do 18: $T_{adv2} \leftarrow \text{Clip}(T_{adv1}^* + \text{Resize}(P))$ 19: for $j \leftarrow 1$ to $\mathcal{T}$ do 20: $O_{adv}, m \leftarrow \mathcal{R}(M, T_{adv2}, \lambda^j)$ 21: $I_{adv} \leftarrow O_{adv}, m + I_{bg}^j (1 - m)$ 22: $\mathcal{L}_{class}^j \leftarrow Conf(I_{adv})_c$ 23: end for 24: $\mathcal{L}_{class} \leftarrow \sum_{j=1}^{ \mathcal{T} } \mathcal{L}_{class}^j$ 25: $\mathcal{L}_{l2} \leftarrow P _2$ 26: $\mathcal{L}_{mix} \leftarrow \mathcal{L}_{class} + \alpha \cdot \mathcal{L}_{l2}$ 27: Update P with $\nabla_P \cdot \mathcal{L}_{mix}$ 28: end for 29: $T_{adv2}^* \leftarrow \text{Clip}(T_{adv1}^* + \text{Resize}(P))$ 30: return $T_{adv}^* \leftarrow T_{adv2}^*$ </pre>
---	--

attack performance in terms of transferability and robustness. In the second part, we evaluate the naturalness of different textures in two-dimensional (2D) and three-dimensional (3D) spaces. We first embed 2D texture patterns into natural background images, then evaluate their semantic consistency using image similarity metrics (SSIM, Wang et al., 2004, FSIM, Zhang et al., 2011 and CSI, Lin et al., 2014). Then, we render objects camouflaged by different textures into a 3D natural scene. Accounting for the effects of viewing angles and occlusion factors on the semantic consistency between textures and the background, we employ Camouflaged Object Detection (COD). COD is a fine-grained object segmentation task (Xiao et al., 2024), specifically designed to identify visually deceptive camouflaged objects in natural environments. By quantifying the impact of different camouflage textures on the performance of COD models, we can indirectly evaluate their naturalness in 3D space. Finally, we perform ablation studies on the proposed method.

4.1.2. Datasets

For fair comparison of attack performance, we follow previous works (Wang et al., 2022, 2021; Zhou et al., 2024b) by collecting realistic vehicle samples through the CARLA simulator (Dosovitskiy et al., 2017). The existing datasets provided by previous works contain a non-negligible amount of samples where the vehicle body is occluded by the buildings, leading to an illogical fusion of the scene and the vehicle during rendering. Therefore, we ensure the vehicle body is fully visible when generating the dataset, collecting 10,938 and 2734 photo-realistic images for training and attack performance evaluation, respectively. During the construction process, the sun elevation angle in the simulation scene was set to either 60° or 90°, while the light intensity multiplier was configured at 0.8, 1.0, or 1.2. Our dataset for the naturalness evaluation comprises two parts: 400 remote-sensing maps directly obtained from the “forest” subset of the RSSCN7 dataset (Zou et al., 2015), and 416 photo-realistic images collected from a jungle

scene⁵ provided by Unity3D (U3D). Note that in the Unity simulation scene, the light source hue was fixed at 41°, with saturation levels set to 0%, 30%, or 60% and light intensity multiplier set to 0.8, 1.0, or 1.2.

4.1.3. Evaluation metrics

Attack success rate (ASR). To evaluate the attack performance, we follow Wang et al. (2022) and Li et al. (2024) to introduce the Attack Success Rate (ASR), which indicates the proportion of samples with missed and false detections to the total number of test samples when the IOU threshold is set to 0.5. An IOU threshold of 0.5 is widely adopted as the qualitative benchmark for determining correct bounding box localization (Redmon et al., 2016). By calculating the rate of prediction failures under this criterion, ASR directly and quantitatively reflects the ability of adversarial textures to evade detection.

Image similarity metrics. To evaluate texture naturalness in 2D space, we use image similarity metrics, including SSIM (Wang et al., 2004), FSIM (Zhang et al., 2011), and CSI (Lin et al., 2014), to measure the semantic consistency between the texture and the background. SSIM measures the similarity of global structural features (luminance, contrast, and structure), while FSIM focuses on low-level local visual characteristics (phase congruency, gradient magnitude). Complementing these, CSI specifically evaluates overall color consistency. Together, these metrics ensure the visual plausibility of camouflage textures across multiple semantic dimensions.

Camouflage success rate (CSR). To evaluate texture naturalness in 3D space, we follow Suryanto et al. (2023) to devise the Camouflage Success Rate (CSR) to quantify the effect of camouflaged objects on the detection performance of COD models. For a labeled region Y and a prediction region $\hat{Y}$, CSR is defined as $\frac{|Y - \hat{Y}|}{|\hat{Y}|}$. The core idea behind this metric is to quantify, at the pixel level, the concealment capability of camouflage textures in complex environments. Its value is positively correlated with how well the camouflaged object blends into the natural scene.

4.1.4. Baseline methods

Among the advanced methods against computer vision, we select DAS (Wang et al., 2021), FCA (Wang et al., 2022), RAUCA-base (Zhou et al., 2024a), and RAUCA-final (Zhou et al., 2024b) as baselines. DAS applies adversarial patches only to the top and sides of the vehicle, whereas FCA generates camouflage textures that cover the entire surface of the object. Both RAUCA-base and RAUCA-final enhance the robustness of full-coverage camouflage against various weather conditions. RAUCA-final further improves upon RAUCA-base by adopting a UV map-based sampling method for rendering, achieving more precise texture generation. All four methods adhere solely to the smooth loss proposed in Mahendran and Vedaldi (2015) to constrain perturbations. Additionally, we include two methods against biological vision: multi-scale camouflage (MsCAM, Pei et al., 2023) and deformation camouflage (DeCAM, Yang et al., 2023). These two methods simulate natural textures through pixelated contours and smoothly closed edges, respectively, yet neither undergoes gradient-based adversarial optimization against DNNs. By conducting joint experiments on these two types of vision camouflage, we can compare the ability of the different methods to balance attack performance and naturalness. In addition, to ensure the reliability and credibility of our results, we include the object’s raw texture (RAW) and a texture with random values (RANDOM).

4.1.5. Parameter settings

During initialization, the dominant color set C is extracted from the U3D jungle scene, and the remaining parameters are shown in Table 1. We use PyTorch3D⁶ to implement differentiable scene rendering. During optimization, we choose YOLOv5m, a widely used version of the

⁵ <https://naturemanufacture.com/forest-environment-set>

⁶ <https://pytorch3d.org>

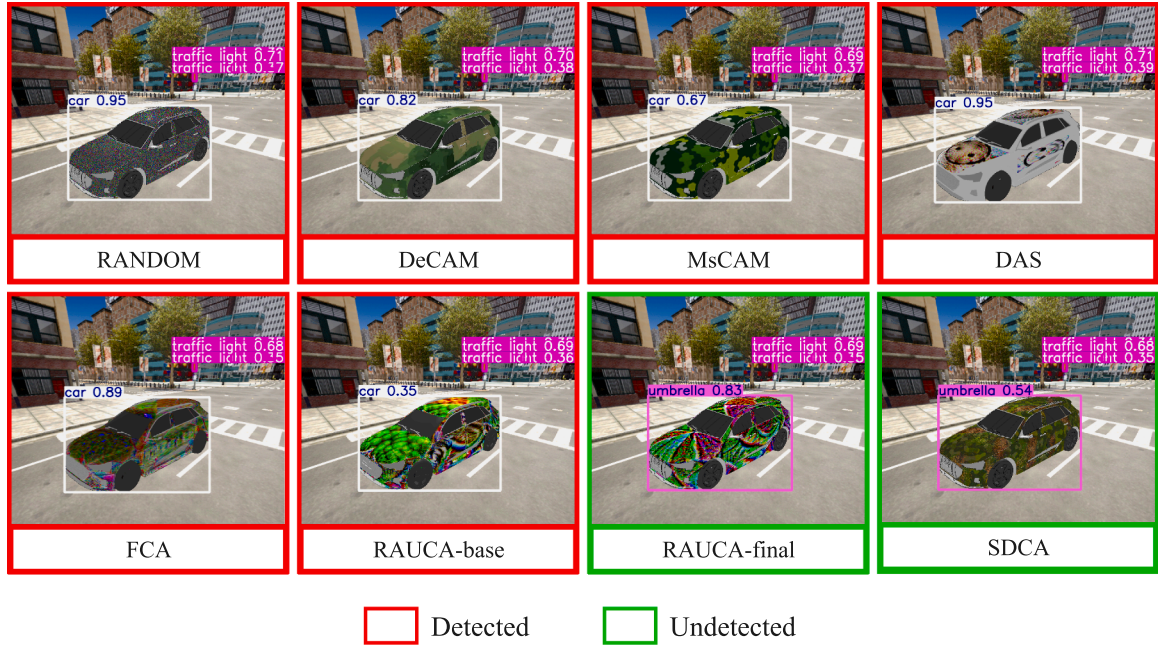


Fig. 7. Visualization of detection results for different textures.

Table 1
Parameter settings for initialization.

Parameter	Value
f_1	10
f_2	20
f_3	40
γ_1	0.57
γ_2	0.19
γ_3	0.14
l_1	2048
l_2	512
ϵ_{pixel}	0.5
σ	0.8

YOLOv5 series,⁷ as our white-box model, setting ϵ_{conf} to 0.05 and ϵ_{IOU} to 0.45. We use the Adam optimizer (Kingma & Ba, 2014) for parameter optimization, with learning rates of 0.05, 0.02, 0.01 for $\mathbf{G}^{f_1}$, $\mathbf{G}^{f_2}$, $\mathbf{G}^{f_3}$, respectively, and 0.001 for P . The value of α is set to 0.0001. In phase one, we constrain the camera distance to the range of [5, 15] to improve model fitting performance. We experiment with an NVIDIA RTX 4090 24GB GPU.

4.2. Attack performance evaluation

4.2.1. Transferability

Transferability across different YOLOv5 versions. Unlike previous works, we first investigate the transferability of existing methods across different versions of YOLOv5. For each camouflage texture, we rendered 301 images, resulting in a total of 2709 images. Fig. 7 illustrates the camouflage effects of different textures, demonstrating that our method effectively deceives the detector into misclassification, achieving a lower confidence score compared to RAUCA-final (Zhou et al., 2024b). Detailed results are presented in Table 2, which indicates that our method not only exhibits superior transferability compared to existing methods, but also demonstrates optimal stability. Notably, despite lacking adversarial optimization, DeCAM (Yang et al., 2023) achieves superior performance

Table 2
Attack performance on different model variants.

Method	ASR(%) [↑]					Average
	v5n	v5s	v5m	v5l	v5x	
RAW	51.50	29.24	24.25	26.91	26.25	31.56
RANDOM	77.74	74.42	55.81	47.18	49.17	60.80
DeCAM	75.42	49.83	28.57	36.21	42.86	46.51
MsCAM	94.02	87.04	52.16	75.42	81.73	78.07
DAS	55.15	42.19	23.59	22.92	30.23	34.88
FCA	79.73	86.05	73.09	61.13	71.10	74.09
RAUCA-base	98.34	97.67	96.68	85.05	96.01	94.68
RAUCA-final	97.67	96.35	97.34	94.02	97.01	96.35
SDCA	99.67	99.67	98.34	97.67	99.67	99.00

Table 3
Attack performance on different model architectures.

Method	ASR(%) [↑]				Variance [↓]
	YLV8	FrRCNN	EfDet	DETR	
RAW	34.22	24.58	35.88	35.88	22.10
RANDOM	83.72	48.84	77.08	58.80	194.53
DeCAM	63.12	38.87	56.81	50.17	80.59
MsCAM	92.69	67.77	85.38	79.07	83.58
DAS	45.18	38.87	45.51	26.91	56.70
FCA	73.75	73.75	83.72	77.74	16.64
RAUCA-base	98.67	84.05	99.67	92.03	39.04
RAUCA-final	99.00	87.04	98.67	93.36	23.65
SDCA	99.34	89.37	95.68	91.03	15.37

over FCA (Wang et al., 2022) and DAS (Wang et al., 2021), revealing that anti-biological vision camouflage inherently possesses adversarial properties capable of disrupting target features. This experimental outcome further validates the correctness of our approach in utilizing natural semantics to initialize texture.

Transferability across diverse detection architectures. For this experiment, we use various models, including YOLOv8 (YLV8),⁸ Faster-RCNN

⁷ <https://github.com/ultralytics/yolov5>

⁸ <https://github.com/ultralytics/ultralytics>

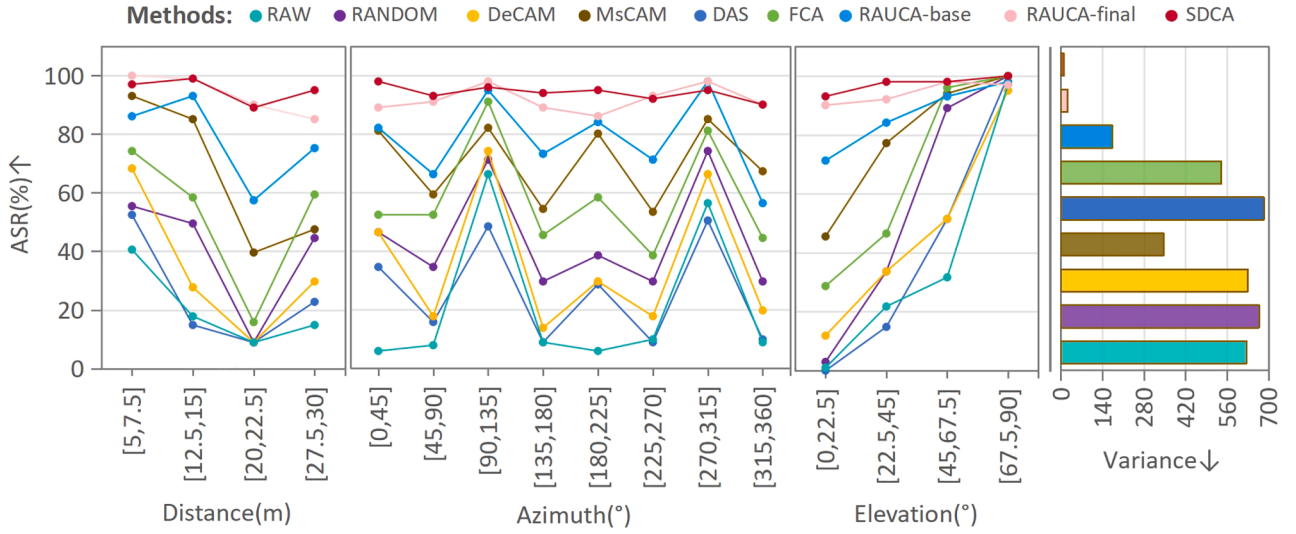


Fig. 8. Attack performance under different camera poses.

Table 4
Attack performance in the real world.

Method	ASR(%)↑					
	YLv5	YLv8	FrRCNN	EfDet	DETR	Average
RAW	54.17	50.00	4.17	54.17	16.67	35.84
SDCA	87.50	75.00	45.83	72.50	50.00	66.17

(FrRCNN, Ren et al., 2016), EfficientDet (EfDet, Tan et al., 2020), and Detection Transformer (DETR, Carion et al., 2020 as black-box models. The experimental results are shown in Table 3. On the one hand, SDCA continues its strong performance on YOLOv8 as it did on YOLOv5. On the other hand, SDCA also exhibits strong performance on completely unknown model architectures. Notably, despite FrRCNN's superior precision compared to one-stage detectors, our method still achieves the highest score. Since SDCA relies on imperceptible discrete perturbations to achieve adversarial attacks, its ability to deceive models with strong global feature extraction capabilities, such as EfficientDet and DETR, is slightly reduced compared to RAUCA-base (Zhou et al., 2024a) and RAUCA-final (Zhou et al., 2024b). Nevertheless, SDCA still demonstrates the most stable attack performance across diverse detection architectures.

Transferability to the real world. Following Suryanto et al. (2023), we evaluate the transferability of SDCA to the real world. To simulate a realistic car paint job, we wrap the camouflage texture around the surface of a car model. We sampled images at different viewpoints and collected a total of 192 images, selecting YOLOv5 (YLv5), YOLOv8 (YLv8), Faster-RCNN (FrRCNN), EfficientDet (EfDet), and DETR as the black box models. Detailed experimental results are shown in Table 4.

4.2.2. Robustness

Robustness under different camera poses. For each camouflage texture, we collected 1616 images covering different camera poses, for a total of 14,544 images. Fig. 8 shows that our method outperforms existing methods under most viewing conditions while demonstrating superior stability (with ASR variance at a single-digit magnitude). Notably, when the azimuth falls within (90°, 135°) or (270°, 315°), or when the elevation increases toward the vertical direction (67.5°, 90°), the difference in ASR values among various camouflage textures diminishes sharply. This occurs because, under these front-view, rear-view, and top-down perspectives, the vehicle's projection approximates a simple rectangle.

Table 5
Attack performance under different occlusion levels.
Values are ASR (%).

Occlusion	dist(m)			
	5~7.5	12.5~15	20~22.5	27.5~30
20%	100	89.11	85.15	94.06
40%	100	90.10	82.18	98.02
60%	98.02	92.08	88.12	99.01
80%	100	96.04	97.03	99.01

This simplified geometry contains significantly fewer discriminative features (e.g., visible wheels, windshields), leading to a highly sparse feature representation. Consequently, the detector has limited information to confidently distinguish vehicles from other objects. Under these conditions, we hypothesize that the effective decision boundary for the vehicle class is less complex. As a result, the feature representations of camouflaged vehicles, which are already close to this boundary due to their ambiguity, become highly susceptible to being pushed across it by even minor perturbations, resulting in misclassification. As a result, the attack performance of different methods shows a convergent trend.

Robustness under different occlusion conditions. Considering the frequent occlusion of vehicles in real-world detection scenarios, we introduced varying occlusion levels (20%, 40%, 60%, 80%) to vehicle bodies and conducted tests at different viewing distances (5~7.5m, 12.5~15m, 20~22.5m, 27.5~30m). For each combination of the occlusion level and the detection distance, we rendered 101 images, for a total of 1616 images. The experimental results are presented in Table 5. For close-range detection (Column 1), our method achieves an ASR of approximately 100%. In medium-range detection (Columns 2–3), due to the reduced vehicle surface area, the ASR shows a moderate decline yet maintains above 82%. Notably, the ASR rebounds in long-range detection (Column 4), which occurs because the detector's accuracy declines when the vehicle appears smaller. Similarly, when the degree of occlusion becomes excessive, the rebound in ASR across most detection distances should not be simply attributed to an enhancement in the camouflage texture's adversarial capability. In essence, severe occlusion may lead to the loss of key discriminative features of the vehicle, causing the detector to miss the object due to insufficient recognizable information.

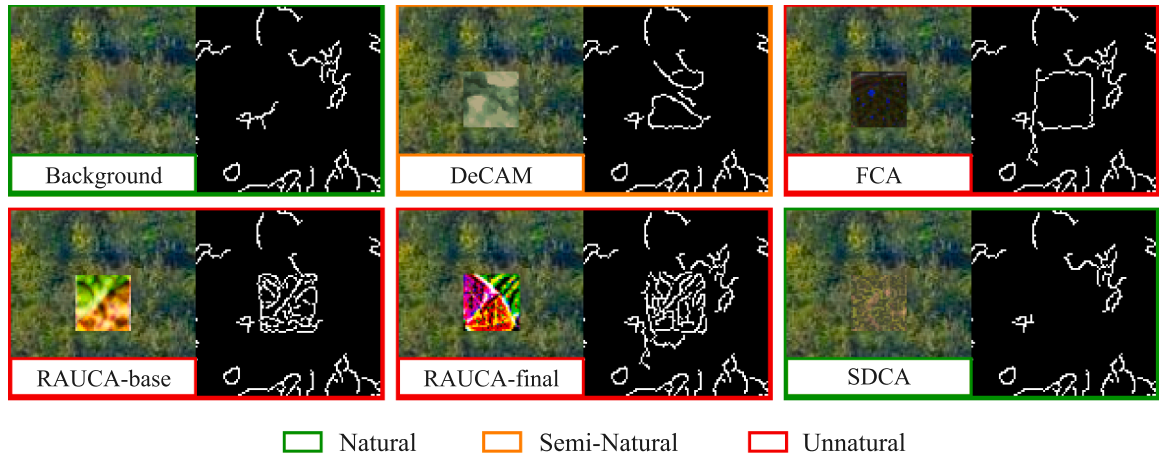


Fig. 9. Visualization of the effect of different textures on the background contours.

Table 6
Effect of texture embedding on image similarity metrics.

Methods	SSIM $\uparrow$		FSIM $\uparrow$		CSI $\uparrow$	
	U3D	RSSCN7	U3D	RSSCN7	U3D	RSSCN7
RAW	0.7096	0.6957	0.8013	0.7874	0.7229	0.6818
RANDOM	0.7284	0.7191	0.9252	0.9275	0.6963	0.6789
DeCAM	0.7487	0.7395	0.9084	0.9092	0.7176	0.7102
MsCAM	0.7328	0.7163	0.8938	0.8838	0.7025	0.6983
DAS	0.7446	0.7406	0.9201	0.9237	0.7230	0.6943
FCA	0.7535	0.7218	0.9193	0.9236	0.7223	0.6896
RAUCA-base	0.7164	0.7055	0.9127	0.9098	0.6950	0.6778
RAUCA-final	0.7055	0.6873	0.8846	0.8737	0.6855	0.6687
SDCA	0.7628	0.7549	0.9310	0.9375	0.7297	0.7069

Table 7
Naturalness of objects camouflaged by different textures. Values are CSR (%).

Methods	SINet-V1		SINet-V2		PFNet	
	near	far	near	far	near	far
RAW	93.60	91.62	95.79	86.02	17.11	72.61
RANDOM	85.99	72.30	92.64	63.47	29.91	81.19
DeCAM	97.19	92.46	95.60	91.84	33.20	79.43
MsCAM	96.86	93.15	92.60	82.56	16.11	79.19
DAS	86.15	65.33	92.75	65.64	27.96	75.51
FCA	90.68	92.03	83.38	69.01	26.43	79.16
RAUCA-base	85.45	87.54	67.38	62.25	27.71	79.00
RAUCA-final	75.70	62.48	87.57	55.01	20.11	77.15
SDCA	97.31	93.16	98.09	95.12	43.43	81.61

4.3. Naturalness evaluation

4.3.1. Naturalness in 2D space

Following Pei et al. (2023), Wang et al. (2023), and Yang et al. (2023), we first embed the camouflage texture patterns (randomly scaled to 0.05-0.125 of their original dimensions) at random positions into natural background images. Subsequently, we perform edge detection (Rong et al., 2014) on the embedded regions and their surrounding areas to evaluate the textures' effectiveness on the structural contours of background elements. Natural background images are sourced from the U3D jungle scene dataset and the forest-class subset of the RSSCN7 dataset. Fig. 9 shows that our method has almost no effect on the structural contours of background elements, achieving high naturalness. Although DeCAM (Yang et al., 2023) alters the original contour structures, its integration of semantic features preserves perceptual naturalness in detection results. FCA (Wang et al., 2022) generates textures with gradient characteristics matching the background's structural contours, but demonstrates poor boundary blending capability with the background. Both RAUCA-base (Zhou et al., 2024a) and RAUCA-final (Zhou et al., 2024b) significantly disrupt the original structural contours and exhibit noticeable semantic inconsistency.

To quantify the above differences, we introduce several image similarity metrics, SSIM (structural similarity, Wang et al., 2004), FSIM (feature similarity, Zhang et al., 2011), and CSI (color similarity, Lin et al., 2014). As shown in Table 6, our method outperforms existing methods across all evaluation metrics. An elevated SSIM score indicates that SDCA maintains high similarity with the background in terms of luminance, contrast, and structure. The superior FSIM score further demonstrates its preservation of key visual features (e.g., phase consistency and gradient patterns), aligning with human perception. Additionally, a higher CSI score confirms SDCA's consistency with the background

color. Notably, FCA exhibits comparable performance to SDCA across all metrics, attributed to its relatively natural texture pattern.

4.3.2. Naturalness in 3D space

In this experiment, we render cars camouflaged by different textures into the U3D jungle scene. For each texture, we render 202 images at stratified distances: near-range (5~15m) and far-range (20~30m), yielding 1818 images total. We select widely-used COD models SINet-V1 (Fan et al., 2020), SINet-V2 (Fan et al., 2021), and PFNet (Mei et al., 2021) as black-box models. Fig. 10 reveals distinct performance between two visual camouflage categories under COD detection: (1) Anti-computer vision camouflage methods (e.g., RAUCA-base, Zhou et al., 2024a and RAUCA-final, Zhou et al., 2024b) fail to evade detection due to insufficient texture-background similarity; (2) Anti-biological vision camouflage methods (e.g., DeCAM, Yang et al., 2023 and MsCAM, Pei et al., 2023) retain detectable local features despite their relatively higher texture-background similarity. Compared to the above methods, SDCA achieves complete concealment, outperforming both anti-computer vision camouflage methods and anti-biological vision camouflage methods. Thus, we can conclude that the texture generated by SDCA exhibits higher similarity to the background, enabling superior object concealment in natural scenes. Note that when a texture's similarity to the background is too low (e.g., RANDOM), COD models dismiss it as irrelevant, which does not indicate high naturalness.

As shown in Table 7, our method achieves higher CSR scores than existing approaches across all models, even outperforming DeCAM and MsCAM, which are specifically designed for evading biological vision systems. These results further demonstrate the transferability of our approach to highly fine-grained detection tasks. For SINet-V1 and SINet-V2, our method demonstrates significant performance in long-range detection. For PFNet, our method effectively enhances camouflage

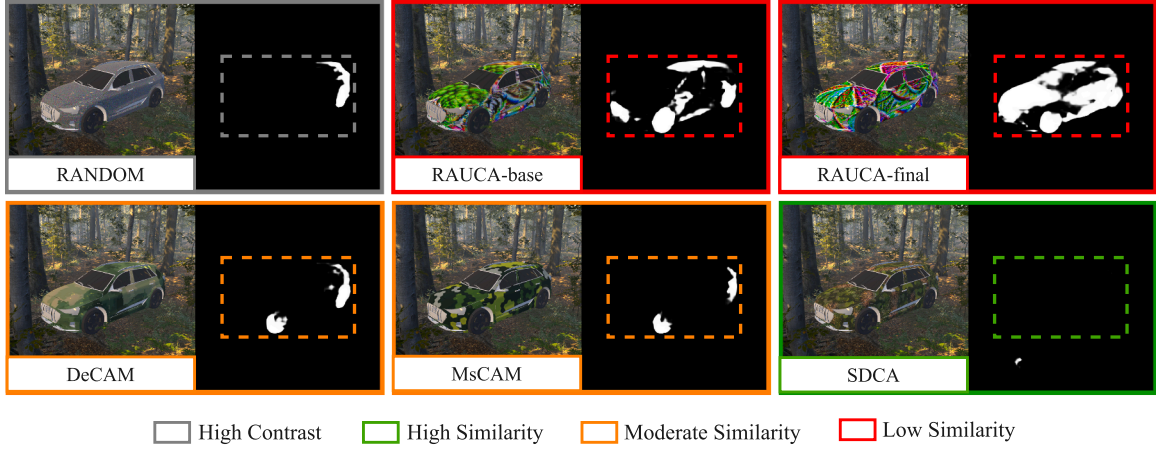


Fig. 10. Visualization of detection results from SINet-V1.

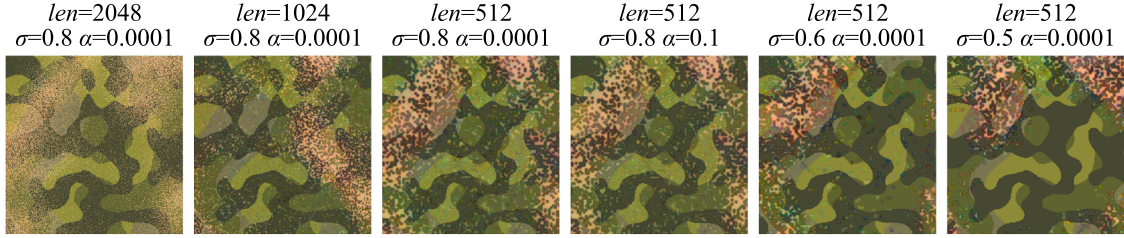


Fig. 11. Visualization of the textures generated with different hyperparameter settings.

Table 8

Effectiveness of textures at different phases. Values are ASR (%).

Texture	YLv8		FrRCNN	
	near	far	near	far
RAW	26.73	8.91	38.61	8.91
SpeCAM	80.20	8.91	72.28	36.63
Phase1CAM	84.16	8.91	76.24	43.56
Phase2CAM	86.14	52.48	93.07	61.39
FinalCAM	94.06	58.42	97.03	72.28

performance in close-range detection. We also observe that PFNet's pyramid feature fusion architecture, coupled with its cross-scale attention mechanism, effectively mitigates missed detections, improving detection accuracy for textures such as RAW and RANDOM.

4.4. Ablation studies

4.4.1. Effectiveness of textures at different phases.

We first evaluate the impact of different texture generation stages on attack performance, using the following textures: original texture (RAW), speckle texture (SpeCAM), texture optimized by phase one (Phase1CAM), texture optimized by phase two (Phase2CAM), and texture optimized by two phases (FinalCAM). The results in Table 8 demonstrate that: (1) SpeCAM and Phase1CAM improve attack performance in close-range detection; (2) Phase2CAM effectively enhances attack capability for long-range detection; (3) FinalCAM achieves the optimal attack performance, confirming that both optimization phases are indispensable.

4.4.2. Effectiveness of different hyperparameter settings.

In this experiment, we evaluate the effectiveness of three key hyperparameters: the side length of the semantic perturbation l_2 , the curva-

Table 9

Effectiveness of different hyperparameter settings.

HyperParams	Performance Metrics				
	ASR(%) $\uparrow$	CSR(%) $\uparrow$	SSIM $\uparrow$	FSIM $\uparrow$	CSI $\uparrow$
Default	98.02	95.86	0.7479	0.9365	0.7095
l_2	1024	93.07	95.17	0.7516	0.9362
	2048	88.12	92.13	0.7532	0.9365
					0.7094
σ	0.5	94.06	96.21	0.7495	0.9362
	0.6	96.04	96.17	0.7492	0.9361
	0.7	98.02	95.69	0.7480	0.9358
	0.9	97.03	95.58	0.7473	0.9364
α	0.1	93.07	93.42	0.7491	0.9355
	0.01	95.05	95.11	0.7489	0.9362
	0.001	97.03	95.39	0.7488	0.9354
					0.7084

ture parameter σ , and the weight of the regularization loss α . A baseline configuration is set with $l_2 = 512$, $\sigma = 0.8$, $\alpha = 0.0001$. As shown in Fig. 11: (1) a larger l_2 enhances the detail representation of the generated texture and effectively reduces the visibility of anomalous pixel values. (2) σ primarily controls the density of discrete points in the sparse regions of the starry texture, with higher values yielding patterns closer to Xingkong camouflage. (3) α has a relatively minor impact on perturbation intensity. The experimental results in Table 9 demonstrate that: (1) Increased l_2 and α degrade attack performance and naturalness. (2) σ mainly affects the trade-off between attack performance and naturalness. (3) All hyperparameters demonstrate a negligible impact on image similarity metrics. Given that different hyperparameter settings have little effect on the naturalness of the texture, we prioritized attack performance when generating the texture.

5. Conclusion

In this paper, we propose SDCA, a semantic-guided camouflage generation framework that leverages semantic features of natural textures

(e.g., color distributions and contour structures) to optimize adversarial textures for evading human eyes and object detectors. Specifically, we first utilize a Semantic-Driven Generator (SDG) to construct initial textures incorporating natural semantic features. Then, we use the Semantic-Constrained Optimization (SCO) strategy to optimize the adversarial texture under semantic constraints. We conduct extensive comparisons with several advanced methods, including DAS (Wang et al., 2021), FCA (Wang et al., 2022), RAUCA-base (Zhou et al., 2024a), and RAUCA-final (Zhou et al., 2024b). While evaluating attack performance, we perform the first comprehensive assessment of camouflage texture naturalness. The results demonstrate that our approach maintains competitive transferability and robustness while preserving superior naturalness.

6. Future work

While this study has yielded encouraging results, several limitations remain. First, as the core objective was to explore the visual concealment of adversarial textures, the training process did not fully account for the impact of real-world physical factors, such as lighting, weather, and sampling noise. In future work, it will be essential to integrate neural renderers capable of simulating realistic physical transformations into the existing framework and to investigate the balance between texture naturalness and robustness to the aforementioned physical variations.

Additionally, although procedural noise can generate relatively natural texture patterns, it is inherently based on randomly generated static textures, which may not adapt well to dynamically changing environments or scenarios requiring specific patterns. To address this, it may be beneficial to combine such methods with advanced generative techniques, such as StyleGAN (Karras et al., 2019) or conditional diffusion models (Mou et al., 2024), to create adversarial textures that can be precisely customized for a wider range of scenarios.

CRedit authorship contribution statement

Haoqin Yuan: Writing – original draft, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Xianyi Chen:** Supervision, Resources, Project administration, Funding acquisition, Formal analysis; **Qi Cui:** Writing – review & editing, Writing – original draft, Methodology; **Fazhan Liu:** Visualization, Validation, Software, Data curation; **Zhangjie Fu:** Resources, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Abiyev, R., & Arslan, M. (2023). Vehicle detection systems for intelligent driving using deep convolutional neural networks. *Discover Artificial Intelligence*, 3(1), 16.

Army Recognition (2019). China: features of PLA's latest combat camouflage uniforms. <https://www.armyrecognition.com/focus-analysis-conflicts/army/defence-security-industry-technology/china-features-of-pla-s-latest-combat-camouflage-uniforms>.

Aurenhammer, F. (1991). Voronoi diagrams—a survey of a fundamental geometric data structure. *ACM Computing Surveys (CSUR)*, 23(3), 345–405.

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In *European conference on computer vision* (pp. 213–229). Springer.

Caro, T., & Koneru, M. (2021). Towards an ecology of protective coloration. *Biological Reviews*, 96(2), 611–641.

Chiu, H.-K., Hachiuma, R., Wang, C.-Y., Smith, S. F., Wang, Y.-C. F., & Chen, M.-H. (2025). V2V-LLM: Vehicle-to-vehicle cooperative autonomous driving with multi-modal large language models. [arXiv:2502.09980](https://arxiv.org/abs/2502.09980).

Cuthill, I. C. (2019). Camouflage. *Journal of Zoology*, 308(2), 75–92.

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). Carla: An open urban driving simulator. In *Conference on robot learning* (pp. 1–16). PMLR.

Fan, D.-P., Ji, G.-P., Cheng, M.-M., & Shao, L. (2021). Concealed object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10), 6024–6042.

Fan, D.-P., Ji, G.-P., Sun, G., Cheng, M.-M., Shen, J., & Shao, L. (2020). Camouflaged object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 2777–2787).

Gustavson, S. (2005). Simplex noise demystified. Linköping University, Linköping, Sweden, Research Report 1, 6.

Jia, Q., Xu, W. D., Hu, J. H., Liu, J., Yang, X., & Zhu, L. Y. (2020). Design and evaluation of digital camouflage pattern by spot combination. *Multimedia Tools and Applications*, 79(29), 22047–22064.

Jiang, P., Ergu, D., Liu, F., Cai, Y., & Ma, B. (2022). A review of yolo algorithm developments. *Procedia Computer Science*, 199, 1066–1073.

Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 4401–4410).

Kato, H., Ushiku, Y., & Harada, T. (2018). Neural 3D mesh renderer. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 3907–3916).

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).

Lagae, A., Lefebvre, S., Cook, R., DeRose, T., Drettakis, G., Ebert, D. S., Lewis, J. P., Perlin, K., & Zwicker, M. (2010). A survey of procedural noise functions. In *Computer graphics forum* (pp. 2579–2600). Wiley Online Library (vol. 29).

Li, Y., Peng, X., & Lian, Z. (2024). Multi-texture fusion attack: A robust adversarial camouflage in physical world. In *International conference on intelligent computing* (pp. 186–198). Springer.

Lin, C. J., Chang, C. C., & Lee, Y. H. (2014). Developing a similarity index for static camouflaged target detection. *The Imaging Science Journal*, 62(6), 337–341.

Ma, F., Feng, Y., Zhang, F., & Zhou, Y. (2025). Cloud adversarial example generation for remote sensing image classification. *IEEE Transactions on Geoscience and Remote Sensing*, 63, 1–16. <https://doi.org/10.1109/TGRS.2025.3559027>

Mahendran, A., & Vedaldi, A. (2015). Understanding deep image representations by inverting them. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 5188–5196).

Mei, H., Ji, G.-P., Wei, Z., Yang, X., Wei, X., & Fan, D.-P. (2021). Camouflaged object segmentation with distraction mining. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 8772–8781).

Mou, C., Wang, X., Xie, L., Wu, Y., Zhang, J., Qi, Z., & Shan, Y. (2024). T2i-adaptor: Learning adapters to dig out more controllable ability for text-to-image diffusion models. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4296–4304). (vol. 38).

Pei, H.-N., Wu, M., Huang, X.-Q., Ding, M., & Zhang, X.-X. (2023). Maneuvering target oriented digital camouflage image generation method. *Displays*, 76, 102365.

Perlin, K. (1985). An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3), 287–296.

Porter, T., & Duff, T. (1984). Compositing digital images. In *Proceedings of the 11th annual conference on computer graphics and interactive techniques* (pp. 253–259).

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 779–788).

Ren, S., He, K., Girshick, R., & Sun, J. (2016). Faster r-CNN: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), 1137–1149.

Rong, W., Li, Z., Zhang, W., & Sun, L. (2014). An improved CANNY edge detection algorithm. In *2014 IEEE International conference on mechatronics and automation* (pp. 577–582). IEEE.

Shulman, G. L., Sullivan, M. A., Gish, K., & Sakoda, W. J. (1986). The role of spatial-frequency channels in the perception of local and global structure. *Perception*, 15(3), 259–273.

Smith, A. R. (1978). Color gamut transform pairs. *ACM SIGGRAPH Computer Graphics*, 12(3), 12–19.

Sun, J., Yao, W., Jiang, T., Wang, D., & Chen, X. (2023). Differential evolution based dual adversarial camouflage: Fooling human eyes and object detectors. *Neural Networks*, 163, 256–271.

Suryanto, N., Kim, Y., Kang, H., Larasati, H. T., Yun, Y., Le, T.-T.-H., Yang, H., Oh, S.-Y., & Kim, H. (2022). Dta: Physical camouflage attacks using differentiable transformation network. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 15305–15314).

Suryanto, N., Kim, Y., Larasati, H. T., Kang, H., Le, T.-T.-H., Hong, Y., Yang, H., Oh, S.-Y., & Kim, H. (2023). Active: Towards highly transferable 3D physical camouflage for universal and robust vehicle evasion. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 4305–4314).

Tan, M., Pang, R., & Le, Q. V. (2020). Efficientdet: Scalable and efficient object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10781–10790).

Wang, D., Jiang, T., Sun, J., Zhou, W., Gong, Z., Zhang, X., Yao, W., & Chen, X. (2022). Fca: Learning a 3d full-coverage vehicle camouflage for multi-view physical adversarial attack. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 2414–2422). (vol. 36).

Wang, J., Liu, A., Yin, Z., Liu, S., Tang, S., & Liu, X. (2021). Dual attention suppression attack: Generate adversarial camouflage in physical world. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 8565–8574).

Wang, Y., Fang, Z., Zheng, Y.-f., Yang, Z., Tong, W., & Cao, T.-y. (2023). Dual attribute adversarial camouflage toward camouflaged object detection. *Defence Technology*, 22, 166–175.

Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612.

- Worley, S. (1996). A cellular texture basis function. In *Proceedings of the 23rd annual conference on computer graphics and interactive techniques* (pp. 291–294).
- Xiao, F., Hu, S., Shen, Y., Fang, C., Huang, J., He, C., Tang, L., Yang, Z., & Li, X. (2024). A survey of camouflaged object detection and beyond. [arXiv:2408.14562](#).
- Xue, F., Xu, S., Luo, Y.-T., & Jia, W. (2016). Design of digital camouflage by recursive overlapping of pattern templates. *Neurocomputing*, 172, 262–270.
- Yang, X., Xu, W.-d., Jia, Q., Li, L., Zhu, W.-n., Tian, J.-y., & Xu, H. (2020). Research on extraction and reproduction of deformation camouflage spot based on generative adversarial network model. *Defence Technology*, 16(3), 555–563.
- Yang, X., Xu, W.-d., Liu, J., Jia, Q., Liu, H., Ran, J.-g., Zhou, L., Zhang, Y., Hao, Y.-b., & Liu, C.-c. (2023). A small-spot deformation camouflage design algorithm based on background texture matching. *Defence Technology*, 19, 153–162.
- Zhang, L., Zhang, L., Mou, X., & Zhang, D. (2011). Fsim: A feature similarity index for image quality assessment. *IEEE Transactions on Image Processing*, 20(8), 2378–2386.
- Zhang, Y., Foroosh, P. D. H., & Gong, B. (2019). Camou: Learning a vehicle camouflage for physical adversarial attack on object detections in the wild. In *International conference on learning representations*.
- Zhou, J., Lyu, L., He, D., & Li, Y. (2024a). Rauca: A novel physical adversarial attack on vehicle detectors via robust and accurate camouflage generation. [arXiv:2402.15853](#).
- Zhou, J., Lyu, L., He, D., & Li, Y. (2024b). Toward robust and accurate adversarial camouflage generation against vehicle detectors. [arXiv:2411.10029](#).
- Zou, Q., Ni, L., Zhang, T., & Wang, Q. (2015). Deep learning based feature selection for remote sensing scene classification. *IEEE Geoscience and Remote Sensing Letters*, 12(11), 2321–2325.